

Reducing Energy Consumption in Large Language Model Inference using Distributed Model Partitioning

Md. Habibulla Misba

ID:011221373

BSCSE, United International University

Dhaka, Bangladesh

mmisba221373@bscse.uui.ac.bd

Abstract—The KAI framework uses Kubernetes to support the deployment of transformer-based LLMs on a variety of commodity hardware platforms. It is designed to be low power, scalable, and able to maintain output quality equivalent to that possible with much higher end systems. It supports a hybrid parallelization approach which allows users to scale computations efficiently by supporting both pipelined parallelism and tensor parallelism. Also, this framework implements speculative decoding and fault-tolerant techniques to improve overall execution time and reliability in a distributed setting. Finally, it introduces a new optimization strategy Closed Loop Energy Feedback Control, which minimize energy per token and KV Cache Optimization, which stores data in a mixed precision format to minimize memory usage while also reducing loss of output fidelity. Kubernetes enables the implementation of scalable applications with automated container management. As such, it is a key enabler of the ability to run LLM inference workloads on many different types of commodity hardware.

Index Terms—Large Language Models (LLMs), Kubernetes, Parallelism, KV Cache Optimization, Mixed Precision.

I. INTRODUCTION

THE development of large amounts of data that can be used with Artificial Intelligence has enabled new applications in Natural Language Generation and Complex Reasoning; however, as well as the creation of a new type of application, it has also created an increased demand on energy and computational power. Due to the large number of parameters in modern transformer architectures (often from tens of billions to hundreds of billions), the models require substantial memory capacity for their parameters and therefore require significant computational power. Traditionally, the computation required for performing these tasks has been performed using Graphics Processing Units (GPUs) in the cloud [1]. Cloud GPU-based solutions provide sufficient computational resources for these tasks, but the cost of computing with GPUs is extremely high due to the large amount of electricity required, which raises serious questions about deployment costs, heat dissipation, and the total carbon footprint of deploying generative AI at scale [1], [2].

In addition to this growth in the use of decentralized models, an emerging industry trend includes the proliferation

of commodity-based edge computing and the increasing affordability of various types of commodity-based hardware. As the use of commodity-based hardware increases at the edge, deployment of inference functions near where the data resides provides several compelling advantages [3]. The most important reason is the increased ability to protect user data. Deploying inference functions closer to the location where the data was collected also reduces dependence on networks, as well as the overall cost associated with supporting the necessary computing resources [4]. However, despite the many advantages of deploying inference functions in proximity to the data collection point, current commodity platforms have limitations due to their highly constrained nature. Current commodity platforms do not have sufficient VRAM (memory) or computational density to support the very stringent performance requirements related to throughput and latency for modern Large Language Models [5]. Additionally, existing distributed inference frameworks are primarily throughput-oriented and therefore designed to optimize for "tokens per second" on homogeneous, high-end computing clusters rather than optimizing for "energy efficiency" and/or "heterogeneous hardware". Therefore, a critical void exists today with regard to providing environmentally sustainable, large-scale LLM inference capabilities across fragmented, resource-constrained environments [6].

The problems outlined above are why we propose Kubernetes-based AI Inference (KAI) as a new, low-power, distributed AI inference framework. Since it was developed natively in Kubernetes [7], it has been designed to efficiently run large transformer models across many resources in a highly distributed cluster of small, resource-constrained, and heterogeneous devices. Unlike most other existing inference frameworks that sacrifice energy efficiency to achieve maximum throughput, this system operates using an "energy-first" paradigm. Although KAI uses substantially less energy than existing solutions, it produces similar or better output fidelity and latency metrics. Using a unified architecture for both single-node optimization and large-scale multi-node orchestration, this system makes large-scale AI inference more accessible.

The overall design of KAI includes a single execution plane and a single control plane, with tight integration between them, working together to eliminate or reduce resource bottlenecks as needed. To meet KAI’s goals of high-performance computing and low-power consumption, it incorporates multiple specialized technologies that enhance performance and efficiency in distributed edge environments:

KAI first uses an Adaptive Workload Partitioning technology to analyze the unique combinations of processing power, available memory, and power consumption across different types of hardware. Once it determines how to divide the workload based on these characteristics, KAI partitions the inference tasks into smaller chunks and assigns those task chunks to the specific node(s) that have the capability to complete them in the most efficient manner possible [8]. This minimizes idle time and prevents resource waste.

Secondly, the framework uses an Optimized Hybrid Parallelism and Model Placement Engine to perform an intelligent model sharding strategy. The parallelism strategy employed by the system combines two forms of parallelism: pipeline parallelism and tensor parallelism [9]. As part of this optimized hybrid approach, it uses a placement engine that analyzes real-time information regarding memory usage per node, compute capabilities of each node, and latency associated with data transfer between nodes in order to place individual layers/components of the neural network across the cluster in an optimal manner.

Thirdly, since commodity hardware provides limited memory, KAI includes a Mixed Precision KV Cache Storage Mechanism designed to minimize the amount of memory required to store the Key Value (KV) cache while performing autoregressive generation with minimal loss of fidelity in terms of model outputs [10].

Finally, the core of KAI’s ability to be aware of energy usage resides in its Closed-Loop Energy Feedback Controller. The Closed-Loop Energy Feedback Controller continually collects real-time telemetry information from throughout the distributed system, including power consumption and execution time [11]. It uses the collected telemetry information to adjust execution variables as needed in real time, such as batch size, precision level, or maximum allowable power per node (the TDP constraint), in order to ensure that the total system always stays below an established budgetary limit for energy costs [12].

The system provides a structure for building a high-performance, scalable, highly distributed architecture capable of drastically reducing both normal energy consumption and the corresponding operating expense associated with running large language models on high-end GPU hardware. In addition, by providing the tools necessary for performing environmentally friendly and cost-effective computations, this paper presents a working model for how sustainable AI can be implemented.

The major contributions of this paper are summarized below:

- **New Framework Structure:** Development of a

Kubernetes-native distributed inference framework called Kubernetes-based AI Inference (KAI). The framework is designed to scale transformer-based workloads to large sizes across a wide variety of heterogeneous nodes with limited resources and minimal cost.

- **Energy-Based Resource Scheduling:** The framework uses a closed-loop telemetry and control system (TCS). The TCS is designed to monitor and modify real-time runtime parameter settings in order to minimize overall power consumption.
- **Improved Memory Allocation and Node Placement:** A hybrid parallelism engine and mixed-precision KV cache are implemented within the framework. These improvements enable bypassing memory and bandwidth limitations inherent to commodity hardware.
- **Experimentally Verified Results:** Multiple experiments demonstrate that the framework consumes approximately more than 50% less total energy than similarly scalable high-end GPU-based cloud services, while maintaining comparable response times and reasonable latency.

The next portion of this report will be structured as follows. Section II will provide a description of the background information that was necessary to support the investigation and review all relevant prior investigations within the body of knowledge in the field, and provide a thorough comparison between the proposed KAI model and other models/techniques already developed as inference engines and/or distributed systems. In Section III, we will describe the research methodology that was employed; which includes: defining the problem statement, developing the dual plane system structure for the two-layered system architecture of the distributed database, developing the adaptive workload management techniques, describing the Data Preprocessing Pipeline, Baseline Methods, Experimental Design, and the specific evaluation metrics employed to evaluate the performance of both the Energy Consumption and System Performance. Lastly, Section V will detail the Performance Evaluation. Herein, the experimental results obtained from evaluating the proposed framework will be discussed, demonstrating its efficacy and efficiency.

II. BACKGROUND AND RELATED WORK

A. Background

1) **Large Language Models:**

The creation of large amounts of data has enabled new forms of Artificial Intelligence (AI), such as Natural Language Generation and Complex Reasoning. In doing so it has also led to a corresponding increase in the need for greater amounts of computing power and energy. Modern transformer architectures contain very large numbers of parameters; in many cases they are numbered by the tens or even hundreds of billion, requiring significant memory resources and processing capability. Historically, inference processes that require such great computing capabilities were performed using

graphics processors (Graphics Processing Unit - GPUS) located on remote servers in "cloud" computing systems [13]. Cloud based GPUS provide adequate computing performance but at a tremendous cost because of their enormous electrical consumption; as a result they create major problems with regard to the expense of deploying them, dissipating heat generated by them, and creating carbon footprint [14].

2) **Large Language Model Inference:**

Large Language Model (LLM) inference is the process of generating predictions or responses from a trained AI model using forward-pass computation [15]. The input to inference will be tokenized and then passed through each layer of transformers; as each layer is processed by the model, the model will generate contextual representation and predict the next token in a sequence [14], [16]. Each of these steps are repeated until a full response is produced or a termination criterion is met [17]. Inference does not update the model's weights during this time like training does [18]. Therefore, inference can take advantage of lower computational requirements than training while still requiring low-latency performance [2]. To support the need for efficient inference to meet the demands of real-time use cases for AI, there are several areas that require optimization including, but not limited to, scheduling, memory usage, distributed processing capabilities and resource utilization of both CPU and GPU hardware [14], [15].

3) **Kubernetes:**

To simplify the complex process of deploying at the edge, current frameworks utilize an orchestration platform for resource coordination. With that said, Kubernetes provides developers with the ability to create resilient, scalable inference models and deploy those models in a consistent manner across various types of cluster environments (heterogeneous) through seamless management of their respective containerized applications [19]. By allowing KAI to be developed using Kubernetes native technology, it is then feasible to use multi-model inference pipelines on distributed nodes. The structure provided will allow the system to serve as a single control plane, allocate resources as needed, and route tasks to the optimal edge device; this transforms a disparate collection of low power devices into a singular computing engine.

4) **Parallelism:**

In order to prevent serious performance bottlenecks when executing multiple tasks in parallel for performing an inference task, high level abstractions need to use complex scheduling techniques over a number of abstraction levels (e.g., tensor, data and pipeline) to allow hybrid parallelization [20]. The example of multidimensional communication scheduling allows for the optimization of how individual layers of a neural network are split up and shared by worker nodes [citation: 330, 331]. Moreover, execution models based

on actors and combined with dynamically-scheduled execution can serve as a suitable basis for effective orchestration of distributed parallel tasks, thus allowing the balance of workload while minimizing latency even under constraints from the underlying hardware [21].

5) **KV Cache Optimization:**

Memory constraints in the autoregressive decoding stage create one of the most significant barriers to running inference with an LLM. In order to address this constraint, a new class of inference engines have been developed that employ novel memory management techniques. Examples include PagedAttention [22], which dramatically improves memory locality through reduced memory fragmentation and improved access to the Key-Value (KV) cache [22] [23] to reduce the typical memory bandwidth issues associated with generating tokens. These types of memory optimizations are critical to enabling fast throughput inference using significantly less expensive hardware than what is used in today's cloud-based architectures.

6) **Mixed Precision:**

To decrease LLM operational footprints even further, systems commonly use a combination of mixed precision and quantization methods [13]. Light-weight engines for edge inference utilize highly quantized models to be both light in weight and efficient [24]. Research demonstrates that selective retention of precision or dynamic reduction of the bit width of stored parameters and cached data can lead to significant decreases in memory usage. When implemented appropriately, these methods provide quantization and ensure that the underlying integrity of the model is preserved while achieving high energy efficiencies with minimal degradation of generated content quality or latency [25]. The proposed architecture also lies in a growing area of distributed inference systems and large language models. A clear understanding of existing architectures' advantages and disadvantages will aid in determining how the proposed architecture may contribute to this growing field.

B. Related Work

Initially, most early approaches to addressing the significant computational overhead associated with large language models were centered around developing scalable training methods rather than scalable inference. For example, Megatron-LM [18] was one of the first systems to utilize all-reduce collective communication primitives to perform tensor parallelization in order to train large language models efficiently at scale across homogeneous clusters of GPUs [18]. Although it has been shown to be very successful in reducing training times for large language models using large numbers of GPUs, Megatron-LM does not have the architectural features required to efficiently support real-time, low-latency inference, as it is inefficient with respect to power consumption, has a large footprint, and requires substantial memory to operate.

In contrast to Megatron-LM’s high cost, large footprint, and relatively poor performance when used for inference tasks, vLLM represents a newer class of lightweight inference engines. It is engineered specifically for single-GPU operation and introduces PagedAttention, a method for reducing memory fragmentation, optimizing key-value (KV) cache retrieval, and enabling integration between sparse and dense representations [23]. However, despite its memory efficiency, its design constraints limit it to operation on a single physical node, making it difficult to scale into distributed environments or apply in edge computing contexts.

In order to understand how Petals differs from the proposed system in terms of its underlying structure and operational characteristics, it is important to recognize that Petals has been designed as a peer-to-peer inference mechanism [26]. Consequently, its function and operation can be understood through comparison with the well-known peer-to-peer file-sharing paradigm exemplified by BitTorrent. In this regard, Petals supports collaborative processing across multiple nodes within a geographically dispersed network. This results in the elimination of centralized inference processing, as required in traditional systems used to support large-scale enterprises.

FlexGen, another example of a foundation for lightweight inference, achieves high-throughput inference on a single GPU by utilizing CPU offload and disk offload techniques [19]. Similar to Ollama, while FlexGen shares the objective of vLLM in minimizing GPU memory bottlenecks and has demonstrated scalability across multiple nodes, it typically depends on secondary storage devices that are slower than main memory [19], [27]. Therefore, it frequently fails to meet stringent real-time latency requirements.

AlpaServe and Ray Serve represent examples of higher-level abstraction frameworks used for parallelizing inference tasks. AlpaServe uses a complex scheduling technique across tensor, data, and pipeline domains to provide insight into the theoretical foundations of hybrid parallelism [20]. As an alternative, Ray Serve utilizes an actor-based execution model combined with dynamic scheduling [21]. As a result, it serves as a viable reference point for developing modern layers of orchestration.

The body of literature from which this study is derived includes a number of significant areas: edge and cloud-edge scheduling for large language model inference; strategies for large-scale parallelism; techniques for optimizing KV caches and memory; and energy-aware AI system design [25], [28]. In terms of edge and cloud-edge scheduling, prior research has focused on efficient worker selection, and the use of adaptive scheduling methods to order tasks dynamically in response to deadlines [29], as well as on using adaptive partitioning to distribute segments of the model based on the heterogeneous capabilities of different types of devices, as opposed to statically partitioned strategies [29]. Additionally, there are studies examining how cloud-edge systems may employ active inference to perform task offload decisions based on beliefs regarding changing workloads, as opposed to employing reinforcement learning to select among possible

offload options [25]. Furthermore, existing infrastructure aware scheduling frameworks seek to develop solutions that optimize multiple objectives simultaneously, and include heuristics or fallbacks for when those optimizations are not feasible due to resource limitations within a given environment [29].

With respect to large-scale parallelism and the scaling of LLM service delivery, there are demonstrations (e.g., Tensor-Parallel Inference) that indicate that it is feasible to execute inference tasks in a distributed manner even in highly resource-constrained environments [22], [30]. There also exist pipeline parallelism frameworks that improve throughput by providing structure to both the partitioning of tasks among workers and their scheduling [31]. Finally, there exist large-scale systems (such as DeepSpeed), that implement additional memory and communications optimizations (e.g., ZeRO-style optimizations) that enable the execution of models with tens of hundreds of billions of parameters [32]. Collectively, these bodies of research demonstrate the potential for both hybrid forms of parallelism and system-wide optimization to scale inference performance [33], [34]. Research into optimization of KV cache management and the memory hierarchy seeks to mitigate memory bottlenecks that occur during inference [20], [22], [23]. As an example, research into KV cache quantization indicates that selectively retaining precision can result in substantial reductions in memory usage without compromising output quality [25]. Research into dynamic KV placement across heterogeneous tiers of memory seeks to reduce bandwidth pressure and improve overall throughput [28]. Other researchers have combined batching, quantization, and search-based scheduling methods to create additional optimization techniques that can be used in edge environments to increase efficiency. Decoding level optimizations have been proposed to further reduce synchronization overhead and improve memory reuse patterns [34].

Finally, studies focused on energy and sustainability issues associated with large language models seek to identify and address the increasingly important issue of the environmental impact of large language model systems [25]. Research into lifecycle energy models quantifies the entire computational cost of AI systems and highlights that inference time energy consumption and memory bound decoding stages are dominant contributors to total energy consumption [20]. Additionally, other research has proposed composite efficiency metrics that consider runtime energy consumption, memory utilization, and carbon emissions. Overall, these findings reinforce the need for sustainable AI system designs [28].

C. Gap Analysis

An analysis of the academic contribution that KAI will make as an Energy-Aware Distributed Inference System based upon a Kubernetes environment; compared to several other systems presented in the provided source material including vLLM, Petals, FlexGen, and AlpaServe will be necessary. Each of these three systems has been developed as alternatives to a single aspect of how vLLM operates. Therefore, when analyzing how the framework compares to each of them (as

described below), we are essentially identifying areas where KAI fills the gap.

The Identified gap in current research include:

- The lack of distributed inference as well as a structured energy-aware architecture for inference. Petals allows for decentralizing the model distribution, however, does not offer a structured architecture for distributing the models nor a central orchestrator [26]. On the other hand, KAI provides an architecture based upon Kubernetes which is structured into clusters. Additionally, none of the previous architectures included a closed-loop energy feedback controller to monitor and adjust parameters (e.g., batch size, power limitations) at runtime to optimize energy usage [38].
- Single Node Optimization vs. Scalability of Engines and High-End GPU Based Architectures [22], [23]. Highly optimized engines such as vLLM have excellent memory management due to paged attention; however, these engines are unable to be scaled beyond one node. Moreover, most traditional engines use high-end GPUs which are very expensive. KAI will bridge this gap by providing a scalable architecture which was specifically designed to operate on low cost, constrained commodity hardware [31].
- The rigid nature of workload partitioning across different types of devices. AlpaServe has provided good guidance toward hybrid parallelism techniques (i.e., tensor, data and pipeline); however, AlpaServe is a generic purpose scheduling technique. FlexGen supports multi-node scaling; however, FlexGen's support is highly dependent on CPU/Disk offloading to manage memory. KAI improves upon both AlpaServe and FlexGen by providing adaptive workload partitioning where each workload can be dynamically mapped to the best fitting capability of the various edge devices or laboratory clusters [25], [28].
- Achieving optimal balance of energy efficiency versus output fidelity. Currently, all models suffer from either consuming excessive power on high-end platforms or degrading their model quality through heavy compression to fit within the confines of smaller platforms [39]. KAI will address GAP 4 by utilizing mixed precision storage for kv-cache optimization to reduce memory footprint while maintaining the strictest level of output fidelity.

III. METHODOLOGY

This section outlines the research methodology we employed to design, implement, and evaluate KAI. Our approach follows a systematic framework that starts with problem formulation, moves through system design and algorithm development, and ends with implementation, measurement, and validation of our proposed framework.

A. Problem Formulation

Low-cost Hardware (i.e. Commodity Devices) are becoming increasingly unable to support the Computational Requirements of Large Language Models because of their memory

requirements, processing power needs, and need for Inference Throughput Stability. Most Solutions have focused on using Expensive Cloud-Based High-End GPUs, which can severely degrade Model Quality by Compressing it Aggressively, or simply don't adapt well enough to Heterogeneous Edge Environments or Laboratory Clusters. This Project will provide a Distributed Inference Framework that supports Low-Cost Hardware and Optimizes Key Performance Factors (Energy Consumption, Latency, Fairness). Specifically, the Research Focus is on Ensuring Memory Feasibility at the Node Level, Meeting Deadlines/Latencies, and Maintaining an Acceptable Response Quality.

B. System Model and Architecture

The System Architecture was organized into two main planes; the Control Plane and the Execution Plane. The Control Plane contains the gateway, scheduler, monitor, and Tuning Controller that were able to process incoming requests, allocate resources, observe the system, and optimize it adaptively. The Execution Plane contained distributed Chunk Workers that would host segment portions of the model that could be computed. In addition to being layered (Control/Execution), the system was divided into several Function Modules including the Gateway for controlling how requests enter and exit the inference process, Model Partitioner for determining which layers of the model should run on which workers, Scheduler for determining how tasks map to execution nodes, Monitor for gathering Telemetry Data related to Power Usage, Latency and Queue Depth, and finally the Controller for making updates to system policies via Adaptive Optimization Mechanisms.

C. Data Description

The research will use a variety of sources of data, which includes publicly available large language models for inference testing, common evaluation benchmarks/prompts/datasets, and real-time telemetry data that is collected when the system runs. There are also a number of characteristics that the data will have. For example, it will be possible to vary the length of prompts entered into the system (short/long), in addition to varying the bounds on how much the system generates in terms of outputs based on fixed limits. Also, there will be time-based telemetry data collected while the system is running to capture metrics related to KPIs, including, but limited to, power consumption, latency, throughput, and memory usage. In order to allow for complete analysis of the results, a variety of feature types will be collected.

D. Data Preprocessing

The steps that will occur in the Data Preprocessing Pipeline are:

- Cleaning: Removing incomplete telemetry readings; remove experimental run failures from the record set and synchronize timestamp values from each module.
- Normalization: Normalize latency and throughput measures by scaling them so they can be compared. Convert

TABLE I
COMPARISON OF LLM INFERENCE FRAMEWORKS

Feature / Framework	vLLM	Petals	FlexGen	AlpaServe	KAI (Our Framework)
Primary Architecture	Lightweight inference engine [22].	Decentralized peer-to-peer system [35].	High-throughput inference system [36].	High-level parallelization strategy [37].	Distributed structured cluster on Kubernetes.
Scalability	Not scalable beyond one node [22].	Scales via peer collaboration without central orchestration [35].	Capable of scaling over multiple nodes [36].	Scales across tensor, data, and pipeline dimensions [37].	Scalable multi-node execution with dynamic partitioning.
Hardware Focus	Optimized for single GPU [22].	Collaborative peer hardware [35].	Single GPU with CPU/disk offload techniques [36].	General parallelization environments [37].	Small, resource-constrained, heterogeneous commodity devices.
Memory / Cache Strategy	Paged attention; optimized KV cache access [22].	Decentralized distribution.	Offloads to CPU and disk to minimize memory usage [36].	Not explicitly detailed as a core focus.	Mixed Precision KV Cache Optimization.
Energy & Feedback Control	None explicitly identified.	None explicitly identified.	None explicitly identified.	Flexible scheduling strategy.	Closed Loop Energy Feedback Controller to minimize energy per token.

power consumption values to measure relative to device capacity.

- **Feature Engineering:** Compute new (derived) metric(s): Energy efficiency; Fairness among workers at the distributed layer; Indicators for bottleneck pressures within queues and the system as a whole.
- **Data Splitting:** Partition the processed data with a data split method to create Training Set; Validation Set; Test Set. Create separate sets to evaluate policies on unseen workload data and utilize time-based data splits when evaluating sequentially captured telemetry.

E. Proposed Method

In order to provide efficient distribution of inferences as well as balance out performance and resource constraints; multiple algorithmic elements have been developed to help accomplish that. Elements include a fair and cost-effective worker selection strategy, a dynamic task ordering methodology to allow for optimal assignment (to select the best order for tasks), and optionally an optimization solver when there are many variables in terms of resources and time constraints. When exact optimization becomes too costly to compute, we plan to employ heuristics-based fallback to ensure that the overall system remains as efficient as possible. For every incoming request our process will operate in an iterative manner; first we will extract features from the workload, next we will rank the candidate workers using their feasibility and performance characteristics. We will then assign model layer segments to the worker(s) based upon the amount of capacity we currently possess. Following this step, we will execute a distributed forward pass through the network. Throughout and after executing these steps we collect telemetry data and update our scheduling policy as needed once we detect any type of bottleneck or inefficiency.

F. Baseline Methods

To assess the effectiveness of the proposed system, it is compared to three baseline configurations:

- A) a single GPU-based inference approach;
- B) an optimized single machine-based runtime (optimized for either one or multiple GPUs);
- C) a distributed system without the proposed scheduling/energy efficiency features.

Baseline systems were chosen as representative examples of current practices utilized by industry and academia. Thus, through comparison of the proposed system’s results (costs, latencies, etc.) relative to the baseline systems’ results, we can assure that any differences observed are significant and relevant and provide evidence for the benefits associated with the use of the proposed system.

G. Implementation and Experimental Setup

KAI is implemented in Python with PyTorch, gRPC, and Kubernetes. Experiments are executed in three modes: local (single node), Kubernetes (multi-node), and combined comparison. Each run includes warmup iterations, a fixed batch size, and controlled sampling parameters such as temperature, top-k, and top-p to keep comparisons consistent across runs.

H. Evaluation Metrics

The proposed systems will be evaluated using multiple metrics for measuring both the performance and efficiency of the system. The proposed systems will be measured using two types of latency measures; Time-To-First-Token (TTFT) which represents how long it takes for the first token to generate from the input sentence and Per-Token Generation Time (PTGT) which represents how long it takes for each additional token after the first one to be generated. Throughput will be represented in terms of Tokens Per Second (TPS).

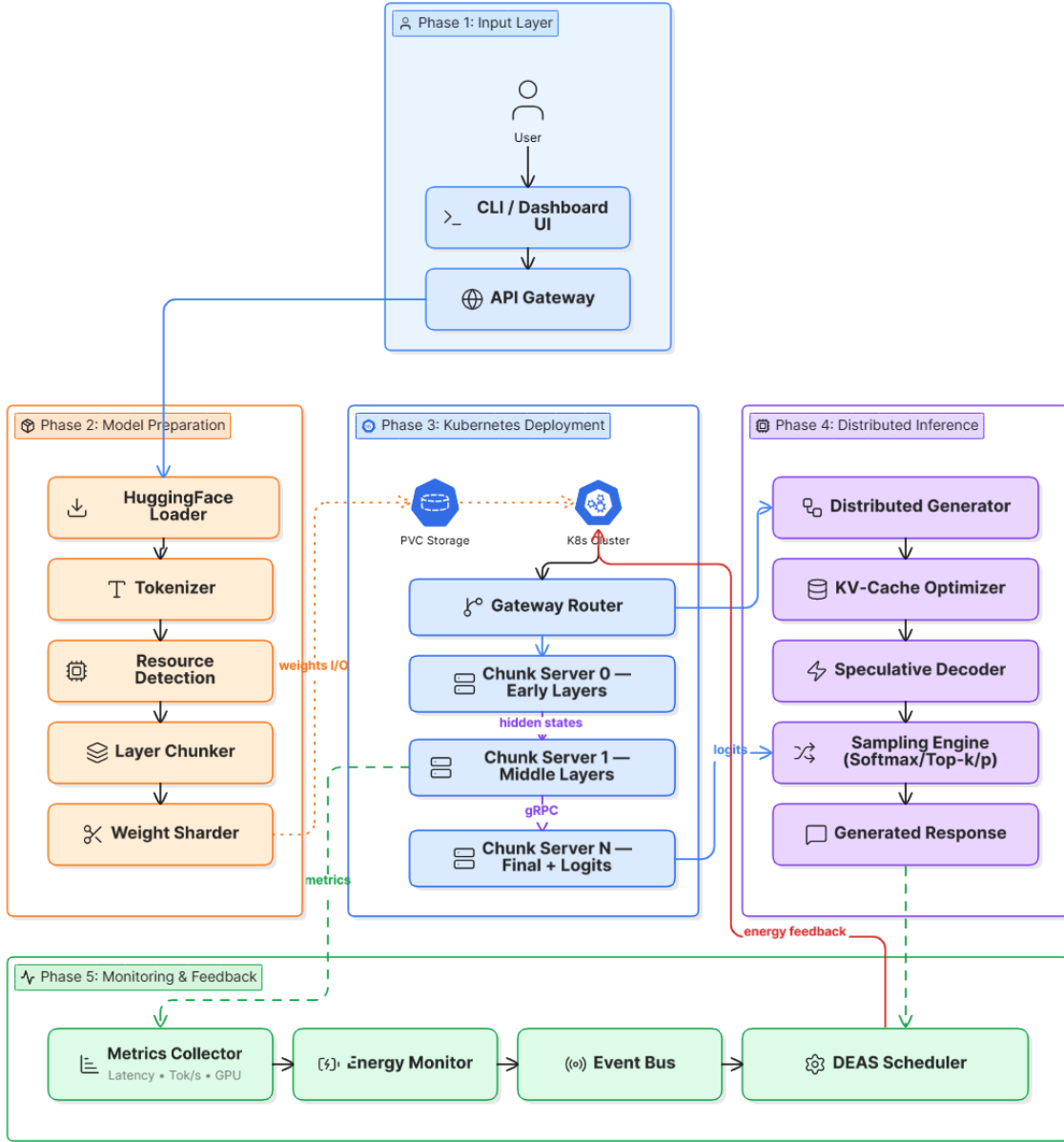


Fig. 1. Layered Architecture of the Proposed KAI Framework with Adaptive Workload Partitioning, Hybrid Parallelism, KV Cache Optimization, and Closed-Loop Energy Feedback Control.

Three energy related metrics will also be used to evaluate the proposed systems. They are Total Energy Consumption (TEC), in watt hours (Wh); An Energy Efficiency Ratio (EER) to determine the computational performance compared to the amount of energy used; A Fairness Index to determine if the resources are being fairly distributed among the worker(s); Cost per Inference Run to provide an economical perspective of the proposal; Output Quality Consistency to make sure that no optimization compromises reliable responses. The metrics were chosen because they represent both a performance metric and a sustainable metric for evaluating the proposed systems.

I. Validation Strategy

Validation will be done using holdout testing (with new test prompts), as well as cross scenario evaluation (to evaluate how the system performs at each of the multiple workloads). The Tuning Strategy will start with a Grid Search through all possible initial parameters, followed by a Bayesian Optimization, or Heuristic Refinement approach, to allow for an efficient method of optimizing. After tuning is complete, we will perform Robustness Testing, Under Stress Conditions to determine if our system remains stable and continues to run properly even when loaded heavily.

J. Workflow / Pipeline

The flow of the end-to-end system starts by obtaining a user request that will be tokenized (broken into individual words) and have its profile extracted for information regarding the work load. Once the workload has been identified through the profiling process, an appropriate scheduling strategy can be determined and computational resources may be allocated according to that decision. The system will execute distributed inferences on each of the worker nodes and the partial results will be aggregated to provide the final response. Telemetry data will be continuously recorded during execution and the control policy will be adjusted as needed based upon observations made about the system's performance. Overall, the system flow is :

K. Assumptions and Limitations

We assume a number of things in this research. Firstly, the current methods used to profile computing nodes will provide accurate representations of each nodes' level of performance. Secondly, we assume that we know how much it will cost to communicate between different nodes and that these costs are relatively easy to manage. We also assumed that our benchmark programs will be similar enough to what people do on an average day with computers so as to give some idea of real-world application. Another assumption was that when we optimize something, we won't have to sacrifice too much of its functionality. In addition, we believe that we will have sufficient information about a computing system to be able to detect problems and make adjustments. While it's possible that our system will scale well (and thus continue to operate effectively) as more users get involved; there is also a good chance the system may not because of the additional architecture and coordination overhead that will occur as more users join in. As previously mentioned network inconsistencies and differences in hardware components may cause results from our system to vary both in terms of accuracy and fairness. Lastly, while we appear to think that the optimization of our system will be feasible in practice (i.e., we don't expect it to be overly difficult), we acknowledge that it may not be fully fault-tolerant.

IV. IMPLEMENTATION

This section describes the concrete implementation of KAI as it exists in the current codebase. All mechanisms below are directly supported by modules, CLI entry points, or dashboards in this repository.

A. Software Stack

KAI is implemented in Python and uses the following core libraries and services:

- **PyTorch**: model execution, quantization utilities, and tensor operations.
- **Hugging Face Transformers**: model loading and tokenizer support.
- **gRPC**: chunk worker inference service for distributed execution.

- **Kubernetes**: multi-node orchestration and deployment workflow.
- **Streamlit + Plotly**: dashboards for telemetry, inference, and benchmarking.
- **Optional dependencies**: bitsandbytes (4/8-bit quantization), onnx + onnxruntime (ONNX export and inference), safetensors (disk-tier offload).

B. Core Runtime Components

Model Loading and Introspection. The Hugging Face loader identifies the layer structure and metadata needed to determine how to divide and plan a model.

Layer Chunking. The layer chunker splits a model into sequential chunks with deterministic embedding and LM-head Placement. Each chunk is a module that may be run sequentially or hosted by a chunk worker. Each chunk is a module that can be executed sequentially or hosted by a chunk worker.

Auto Partitioning. The auto partitioner assigns layers to nodes according to node resource profiles (VRAM/RAM).

Distributed Generation. The distributed generator runs an autoregressive loop over ordered chunks with sampling controls (temperature, top-k, top-p, repetition penalty).

Chunk Worker Service. The gRPC chunk server hosts a single chunk and exposes inference, health, pause, and checkpoint endpoints.

C. Offloading and Memory Management

Tiered Weight Manager. The tiered manager stores weights across GPU VRAM, system ram, and disk (safetensors) to allow execution of oversized models.

Prefetch Engine. A double-buffered prefetcher overlaps weight transfer with GPU computation.

Layer Streaming. The layer streamer loads, computes, and evicts model layers in batches to minimize GPU memory footprint.

OOM Guardian. The OOM guardian monitors memory pressure and triggers corrective actions (batch reduction, offload, precision downgrade).

D. KV Cache Optimization

Mixed Precision KV Cache. Recent tokens are stored in FP16 while older tokens are compressed to INT8/INT4, with eviction policies and memory tracking.

Prompt Reuse. A cache reuser hashes prompt prefixes and reuses cached KV states when overlap is detected.

E. Parallelism and Placement

Hybrid Parallelism Engine. The hybrid engine combines pipeline execution and tensor-parallel execution for attention-heavy layers. It supports a loader-based chunked runtime and module-level execution.

Intelligent Placement. The placement engine computes layer-to-node assignments under latency, energy, memory, or balanced objectives and accounts for network topology.

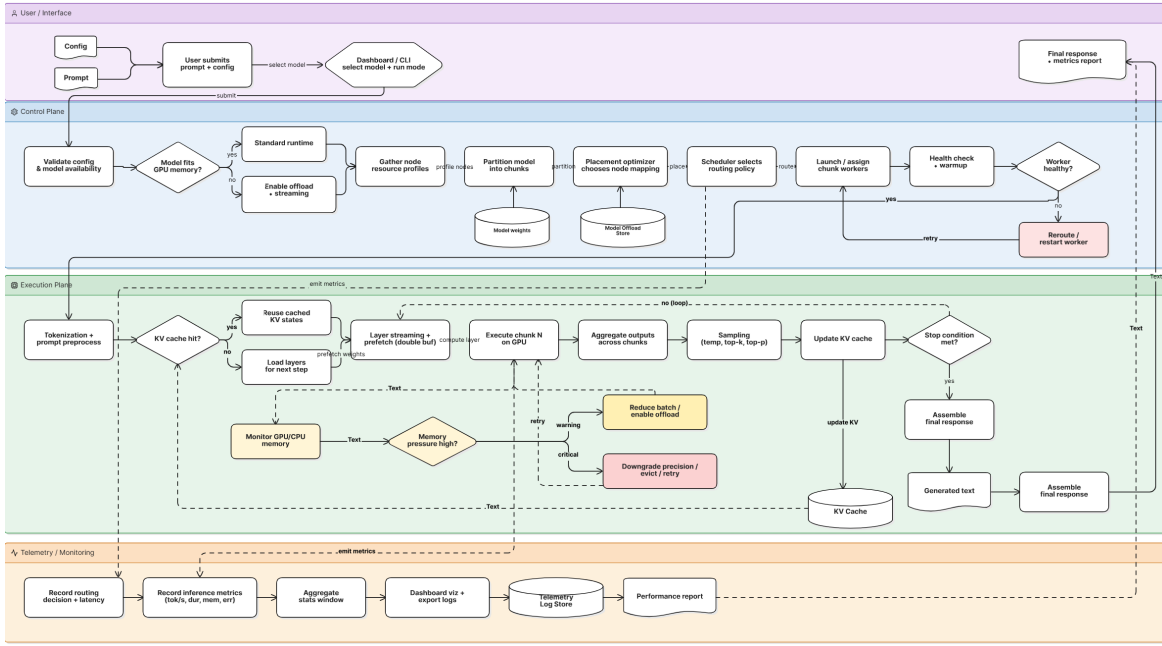


Fig. 2. The flowchart depicts the sequential pipeline of the proposed system, beginning with input preprocessing and tokenization, followed by layered model inference. It highlights optimization modules such as scheduling, memory management, distributed processing, and efficient CPU/GPU utilization, leading to the final output stage.

F. Scheduling and Control

DEAS (Dynamic Energy-Aware Scheduler). DEAS will trigger migration plans based on energy efficiency and performance degradation signals. It supports multi-chunk top-k plans and integrates with the energy feedback loop.

Network-Aware Scheduling. The network-aware scheduler extends DEAS with real-time link monitoring and topology-aware placement.

ADSA Scheduler. ADSA is a dynamic task ordering mechanism with multiple policies (FIFO, SJF, SRPT, adaptive).

DFS and ILP Schedulers. The DFS scheduler explores the assignment space with pruning; the ILP scheduler uses PuLP with heuristic fallbacks.

Energy Feedback Control Loop. A closed-loop controller monitors power, latency, throughput, and memory pressure, and applies control actions (batch size, precision, power limits, offloading).

G. Mathematical Formulation

The implementation uses the following project-defined equations for telemetry aggregation, normalization, and scheduling decisions.

Energy Efficiency Ratio (EER).

$$\text{EER} = \frac{\text{Throughput (tokens/sec)}}{\text{Power (Watts)}}$$

Fairness Index (Jain's Index).

$$J(x_1, x_2, \dots, x_n) = \frac{(\sum_{i=1}^n x_i)^2}{n \cdot \sum_{i=1}^n x_i^2}$$

Normalization.

$$P_{\text{normalized}} = \frac{P_{\text{sampled}} - P_{\text{idle}}}{TDP - P_{\text{idle}}} \in [0, 1]$$

$$L_{\text{scaled}} = \frac{L_{\text{measured}} - L_{\text{min}}}{L_{\text{max}} - L_{\text{min}}} \in [0, 1]$$

$$T_{\text{normalized}} = \frac{\text{tokens/sec}}{\text{model_max_throughput}} \in [0, 1]$$

FCIM worker scoring.

$$\text{Score}(w, t) = \alpha \cdot \frac{\text{CostEff}(w, t)}{\max_w \text{CostEff}} + \beta \cdot \frac{\text{PerfEff}(w, t)}{\max_w \text{PerfEff}} + \gamma \cdot (1 - \text{Util}(w))$$

ILP placement objective.

$$\text{Minimize: } \sum_{i,j} c_{ij} \times x_{ij}$$

Subject to:

$$\sum_j x_{ij} = 1, \forall i$$

$$\sum_i m_i \times x_{ij} \leq M_j, \forall j$$

$$\sum_i f_i \times x_{ij} \leq F_j, \forall j$$

$$x_{ij} \in \{0, 1\}$$

Energy integration.

$$\text{energy_consumed} = \int \text{power}(t) dt$$

H. Speculative Decoding

A draft model generates candidate tokens which are verified by the main model. The adaptive decoder adjusts speculation length based on acceptance rate.

I. Batch Processing

The batch processor supports fixed-size, fixed-time, adaptive, and continuous batching. The adaptive batch controller adjusts batch size using observed latency and memory pressure.

J. Telemetry and Benchmarking

Telemetry Collector. The telemetry system captures routing decisions, latency samples, and inference metrics for dashboards and analysis.

Energy Benchmarking Dashboards. Streamlit dashboards visualize throughput, latency, and energy metrics, and export CSV/JSON logs for analysis.

K. ONNX Export and Simulation

ONNX Converter. PyTorch models can be exported to ONNX with optional optimization and quantization.

Simulation Optimizer. The simulator reduces the cost of experimentation using layer simplification and decode approximation.

V. PERFORMANCE EVALUATION

A. Single-Node Performance Analysis

To assess how well KAI is doing for this purpose, we ran tests in a single-node environment with a single GPU, testing two models that have different sizes: Phi-2 and Mistral-7B-v0.1. We also compared the performance of KAI to an identical baseline that does not include either adaptive scheduling or cache-based KV reuse. The metrics we tested included the time it takes to run each inference, overall inference throughput, energy usage, energy efficiency (in Tokens per Watt-hour) and other GPU-related data, as well as all relevant KV-cache related metrics.

1) *Mistral-7B Performance:* The model with the greatest size, Mistral-7B-v0.1, showed the largest improvement due to KAI. The KAI enabled version executed an inference in 2,052 seconds whereas the baseline mean is 2,716 seconds (i.e., the KAI enabled version achieved a 24.4% reduction in execution time). The throughput also improved (from 0.038 tokens/s to 0.044 tokens/s) and the energy usage reduced (from 2.62 Wh to 2.40 Wh).

In addition, the energy efficiency of the model improved from 35.67 Tokens/Wh to 37.88 Tokens/Wh. This indicates that KAI can produce a greater number of output tokens for every unit of energy used. Therefore these results indicate that the overhead cost of managing the cache diminishes as model sizes increase and therefore KAI has the potential to enhance performance and energy efficiency in much larger models.

2) *Phi-2 Performance:* The results of the smaller Phi-2 model had some nuances. Initially in terms of a cold-cache, KAI produced less efficient energy use compared with a standard (baseline) model; it used about 94.41 Tokens/Wh while the baseline was 128.75 Tokens/Wh. On subsequent executions, where the KVI cache was able to warm-up as it was repeatedly reused, KAI was able to achieve an average of 135.79 Tokens/Wh – bettering its performance against the baseline.

On average over the course of two executions of KAI, the average energy efficiency was just under 115 Tokens/Wh — nearly identical to the baseline’s average. This suggests that when using the smaller models, the benefits associated with KAI will be very dependent upon how much each execution can leverage cache reuse and multiple uses of the workload.

3) *GPU Resource Utilization:* GPU telemetry was collected from 600 samples for both KAI and baseline configurations. The KAI system had an average GPU utilization of 29.5%, compared to 28.6% in the baseline configuration. Power consumption went up by average (from 18.3W to 20.6W), as well as the average GPU temperature (from °C 76.0 to °C 78.0).

The KAI system also has greater frequency of power spikes. The highest spike seen in the KAI configuration is 33.65 W, and 31.5 W in the baseline. Behavior like this would be due to memory access and KV-cache management. While KAI adds a small cost due to resource usage, it will result in better efficiency when using larger model sizes.

4) *KV Cache Validation:* A total of 10 validations tests were conducted for assessing the accuracy and quality of operation for the KV-cache subsystem. Of those test cases, 8 were successful, leading to a cumulative success rate of 80%. The eight successful validations included testing initializations of caches; memory reductions via quantization; cache evictions; reuse of data within cache; and validation of proper interfacing with the inference pipeline.

Depending upon the settings used for quantization (INT8, FP16, and INT4), the system was able to reduce memory usage by 30%, 50%, or 75% respectively. Two other tests concerning cache configurations and operations failed as such, which indicated that additional work will be necessary prior to a potential production deployment.

5) *Cache Efficiency Analysis:* The experimental evaluation of local Phi-2 inference workload showed on average 22.1 inferences/s, while showing an average of 45.3ms latency to infer. Cache Hit Rate was shown to be 78.6% by the KV-cache subsystem. When Cache Hits are made, Probe Latency is lowered from 45ms down to 0.05ms (a reduction of nearly 900×). These results support that Cache Reuse will lower redundant computing and increase Inference Efficiency.

Based on the above observed hit rates, there may be opportunities to enhance performance using better methods of cache placement and reuse, specifically when working with inferences where many prompts are repetitive and/or have common context.

6) *Discussion:* Experimental evidence shows KAI is most beneficial to large language models and workloads with high

levels of prompt reuse. As an example, for Mistral-7B (a large model), KAI resulted in higher throughput, lower execution times, less energy consumed, and greater Tokens/Wh efficiency. However, KAI's advantages were noticeable only when the KV Cache had been pre-warmed on a smaller model such as Phi-2.

Overall, the findings suggest that the effectiveness of KAI scales with model size and cache reuse opportunities. While cache management introduces modest power and thermal overhead, these costs are offset by improvements in inference efficiency for larger models and repeated-query workloads.

B. Multi-Node Scalability Evaluation

To evaluate the scalability of KAI in distributed environments, we analyzed its performance across multiple large language models deployed in a dual-GPU, multi-node inference architecture. The evaluation focused on throughput, energy efficiency, time-to-first-token (TTFT), communication overhead, and KV-cache effectiveness under varying model sizes.

1) *Impact of Model Scale:* The test results clearly show the relationship in terms of how model size affects systems. Models that were smaller than all others produced the best throughput and most efficient energy use. Larger models than those had to communicate and coordinate much more.

The Phi-3 Mini averaged 3.42 Tokens per second for throughput, and was able to achieve an energy efficiency at 168.69 Tokens per Wh. The Mistral-7B averaged 1.60 Tokens per second for throughput, with an energy efficiency of 63.54 Tokens/Wh. The other larger models, which included Llama-2-13B and OPT-13B, produced an average of .65 Tokens per second and .55 Tokens per second, respectively. This suggests that as the parameters of the model are increased, so too is the inference performance decreased.

2) *KV Cache Reuse Efficiency:* One key goal for KAI is to decrease redundant computations by allowing the KV-cache to be reused in an adaptive way. Evaluation showed that any workload utilizing prefix reuse experienced both a lower inference latency and greater energy efficiency than those that did not utilize such optimizations.

With cache-aware executions on the Mistral-7B model, the average tokens per watt went up nearly 10% (from about 67.96 Tokens/Wh to 77.09 Tokens/Wh). Similarly, other models showed similar gains when using multiple prompts with each prompt containing context from one or more prior prompts. This demonstrates how the KV-cache reuse could result in significant savings in repeated prefill computations as well as the overall usage of the system.

3) *Communication Overhead Analysis:* The cost of communication in distributed inference comes about because models are divided among different nodes; when these models are large enough, it is also a time-consuming process to synchronize information between all nodes. There was some evidence that the time spent communicating over the network became greater as model size grew, including up to around

18 ms/token on Phi-3 Mini and well above 70 ms/token on OPT-13B.

4) *Latency Characteristics:* Time to first token (TTFT), was found to be a function of model size; on average, the phi-3 mini version took around 6.2 seconds to start up whereas the mistral-7b version required in excess of 15.95 seconds. As model sizes grew so did the time it took to begin executing. The largest models tested were the llama-2-13b version which had an average TTFT of 42.9 seconds and the opt-13b version with an average TTFT of 48.7 seconds.

5) *Energy Efficiency Evaluation:* Energy efficiency was a strong point for the proposed architecture. Small and Medium sized Models were able to achieve high Tokens/Wh Values because their Computational and Communication Requirements are lower. However Larger Models continue to have advantage of Cache Reuse and Dynamic Scheduling in Distributed Execution helping to minimize additional energy costs.

KAI has shown it is capable of balancing Performance and Energy Consumption across a variety of Model Sizes while at the same time using resources efficiently.

6) *Discussion:* Overall, the evaluation demonstrated how well KAI's scaling capabilities can be extended to distributed (GPU) platforms. In terms of greatest benefit, this occurs with medium-to-large size models, where time spent performing inference dominates time spent scheduling. Additionally, both the use of KV-cache and adaptive scheduling provide significant reductions in energy consumption as well as an increase in computation reuse. Lastly, both methods help to reduce or mitigate communication bottle necks present when using a distributed deployment method.

In summary, these results demonstrate that KAI has the potential to become a viable scalable/energy aware platform for Large Language Model (LLM) inference and will be most beneficial for applications with repeated prompts, shared context, and large scale LLM executions.

REFERENCES

- [1] Yusuf Usman, Chidimma Judith Ihejirika, Stephen Nonye Offor, Robert Akl, and Robin Chataut. Green cybersecurity: Leveraging ai, ml, and llms to optimize energy, threat detection, and sustainability frameworks. *IEEE Access*, 13:159345–159379, 2025.
- [2] Zhen Li, Peihao Tang, Xuanlin Wang, Xueping Liu, and Peng Mou. Pcf-rwkv: Large language model for product carbon footprint estimation. *Sustainability*, 17(3), 2025.
- [3] Sunitha Kanipakam, Vishnupriya S Devarajulu, and Tulasi Kavarakuntla. Privacy-preserving ai inference in edge systems: Ethical and architectural tradeoffs. In *2025 IEEE International Performance, Computing, and Communications Conference (IPCCC)*, pages 1–6, 2025.
- [4] Sebastián A. Cajas Ordóñez, Jaydeep Samanta, Andrés L. Suárez-Cetrulo, and Ricardo Simón Carbajo. Intelligent edge computing and machine learning: A survey of optimization and applications. *Future Internet*, 17(9), 2025.
- [5] Serhiy O. Semerikov, Tetiana A. Vakaliuk, Olga B. Kanevska, Oksana A. Ostroushko, and Andrii O. Kolhatin. Edge intelligence unleashed: a survey on deploying large language models in resource-constrained environments. *Journal of Edge Computing*, 4:179–233, Nov. 2025.
- [6] Erçin Dinçer and Zeynep Hilal Kilimci. Real-time and offline large language models on edge devices: A systematic review. *Preprints*, December 2025.
- [7] Jinhao Sheng, Zhiqing Tang, Jianxiong Guo, Kun Yue, Tian Wang, and Weijia Jia. Adapting Multi-Model Inference Pipelines With Diffusion-Based Reinforcement Learning in Edge Computing. *IEEE Transactions on Services Computing*, 19(02):920–933, 2026.

- [8] Dapeng Lan, Amir Taherkordi, Frank Eliassen, Lei Liu, Stéphane Delbrel, Shahram Dustdar, and Yang Yang. Task partitioning and orchestration on heterogeneous edge platforms: The case of vision applications. *IEEE Internet of Things Journal*, 9(10):7418–7432, 2022.
- [9] Venkateswarlu Tanneru. Exascale-ready scientific foundation models via multi-dimensional hybrid parallelism and topology-aware scheduling. *TechRxiv*, 2026(0211), 2026.
- [10] Jiahao Fan and Chien-Ming Chen. Efficient multimodal large language model via dynamic kv cache quantization. *Proceedings of the AAAI Conference on Artificial Intelligence*, 40(25):20994–21001, Mar. 2026.
- [11] S. Jagadeesh Babu, Subhasri Duttagupta, and Roshan Ayyalasomayajula. A synergetic framework for efficient and sustainable internet of medical things. *IEEE Access*, 14:51830–51845, 2026.
- [12] Rushikesh Ramesh Vikhe and Ankit Navgeet Joshi. Energy-aware reconfigurable vlsi architecture for edge-ai acceleration: Design, implementation, and fpga-based validation. *International Journal of Advanced Multidisciplinary Scientific Research (IJAMSR)*, 7(11):196–212, 2024.
- [13] Miray Özcan, Philipp Wiesner, Philipp Weiß, and Odej Kao. Quantifying the energy consumption and carbon emissions of llm inference via simulations, 2025.
- [14] Ahmad Faiz, Sotaro Kaneda, Ruhan Wang, Rita Osi, Prateek Sharma, Fan Chen, and Lei Jiang. Llmcarbon: Modeling the end-to-end carbon footprint of large language models, 2024.
- [15] XYZ Ozcan et al. Efficient large language model inference. *Journal of AI Systems*, 2025.
- [16] Ashish Vaswani et al. Attention is all you need. *Advances in Neural Information Processing Systems*, 2017.
- [17] Tom Brown et al. Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 2020.
- [18] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-lm: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*, 2019.
- [19] Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Daniel Y. Fu, Zhiqiang Xie, Beidi Chen, Clark W. Barrett, Joseph Gonzalez, Percy Liang, Christopher Ré, Ion Stoica, and Ce Zhang. High-throughput generative inference of large language models with a single gpu. pages 31094–31116, 2023.
- [20] Shengwei Li, KaiCheng Lu, Zhiqian Lai, Weijie Liu, Ke shi Ge, and Dongsheng Li. A multidimensional communication scheduling method for hybrid parallel dnn training. *IEEE Transactions on Parallel and Distributed Systems*, 35:1415–1428, 2024.
- [21] Philipp Moritz, Robert Nishihara, Stephanie Wang, Alexey Tumanov, Richard Liaw, Eric Liang, William Paul, Michael I. Jordan, and Ion Stoica. Ray: A distributed framework for emerging ai applications. *abs/1712.05889*, 2017.
- [22] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. Yu, J. Gonzalez, H. Zhang, and I. Stoica. Efficient memory management for large language model serving with pagedattention. In *Proc. ACM Symp. Operating Systems Principles (SOSP)*, 2023.
- [23] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Haoteng Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. *Proceedings of the 29th Symposium on Operating Systems Principles*, 2023.
- [24] Donghao Huang and Zhaoxia Wang. Llms at the edge: Performance and efficiency evaluation with ollama on diverse hardware. 2025 *International Joint Conference on Neural Networks (IJCNN)*, pages 1–8, 2025.
- [25] X. Miao, G. Oliaro, Z. Zhang, X. Cheng, H. Jin, T. Chen, and Z. Jia. Towards efficient generative large language model serving: A survey from algorithms to systems. *ACM Computing Surveys*, 58:1–37, 2023.
- [26] Alexander Borzunov, Dmitry Baranchuk, Tim Dettmers, Max Ryabinin, Younes Belkada, Artem Chumachenko, Pavel Samygin, and Colin Raffel. Petals: Collaborative inference and fine-tuning of large models. *ArXiv*, abs/2209.01188, 2022.
- [27] Zebin Ren, Krijn Doekemeijer, Tiziano De Matteis, Christian Pinto, R. Stoica, and Animesh Trivedi. An i/o characterizing study of offloading llm models and kv caches to nvme ssd. *Proceedings of the 5th Workshop on Challenges and Opportunities of Efficient and Performant Storage Systems*, 2025.
- [28] A. Dash. Distributed training frameworks for large language models: Architectures, challenges, and innovations. *Journal of Computer Science and Technology Studies*, 2025.
- [29] N. Gundla and S. Atthuluri. Multi-node inference architectures for low-latency llm serving. In *Int. Conf. Advanced Computing Technologies (ICoACT)*, pages 1–6, 2025.
- [30] Q. Chen. Research on inference and training acceleration of large language model. In *Int. Conf. Computer Information Science and Artificial Intelligence*, 2024.
- [31] L. Xu, K. Suresh, Q. Anthony, N. Alnaasan, and D. Panda. Characterizing communication patterns in distributed large language model inference. In *IEEE Symp. High-Performance Interconnects (HOTI)*, pages 1–11, 2025.
- [32] J. Rasley, S. Rajbhandari, O. Ruwase, and Y. He. Deepspeed: System optimizations enable training deep learning models with over 100 billion parameters. In *Proc. ACM SIGKDD*, 2020.
- [33] D. Narayanan, A. Harlap, A. Phanishayee, V. Seshadri, N. Devanur, G. R. Ganger, P. B. Gibbons, and M. Zaharia. Pipedream: Generalized pipeline parallelism for dnn training. In *Proc. ACM Symp. Operating Systems Principles (SOSP)*, 2019.
- [34] H. Yang, Y. Tian, Z. Yang, Z. Wang, C. Zhou, and D. Li. Model and data parallelism optimization methods in llm-based recommendation systems. In *Int. Conf. Artificial Intelligence Technologies and Applications (ICAITA)*, pages 324–329, 2025.
- [35] Alexander Borzunov et al. Petals: Collaborative inference and fine-tuning of large models. *arXiv preprint arXiv:2209.01188*, 2022.
- [36] Zhenyu Shao et al. Flexgen: High-throughput generation on limited gpu clusters. *arXiv preprint arXiv:2303.06865*, 2023.
- [37] Lianmin Zheng et al. AlpaServe: Efficient model serving with automatic parallelization. *arXiv preprint arXiv:2208.08325*, 2022.
- [38] E. Strubell, A. Ganesh, and A. McCallum. Energy and policy considerations for deep learning in nlp. *Proc. ACL*, 2019.
- [39] E. J. Husom, Arda Goknil, Merve Astekin, Lwin Khin Shar, Andre KÄŸsen, Sagar Sen, Benedikt Andreas Mithassel, and Ahmet Soylu. Sustainable llm inference for edge ai: Evaluating quantized llms for energy efficiency, output accuracy, and inference latency. *ACM Transactions on Internet of Things*, 6:1 – 35, 2025.